BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
II SEMESTER 2018-2019
EEE/CS/INSTR F241 MICROPROCESSOR PROGRAMMING AND INTERFACING
Comprehensive Exam (OPEN BOOK)

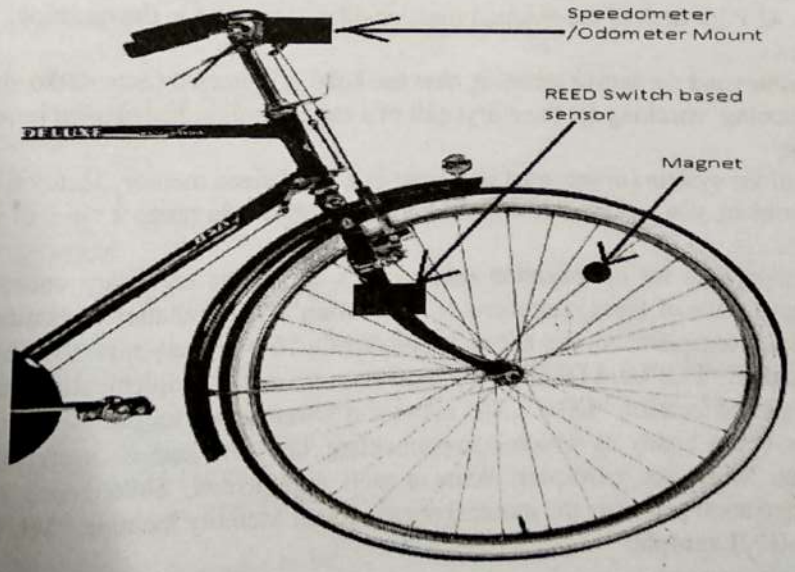
M.M:80 03-05-2018 DURATION: 2hrs

Note: Attempt all parts of the questions at one place only

Q1. Design a bicycle odometer using 8086 microprocessor. Odometer keeps a record of total distance travelled by the bicycle (in Km). There is a magnet and magnetic reed switch is mounted on the wheel spokes and wheel fork respectively as shown in the illustration. Each time magnet passes through the Wheel fork , magnetic reed switch closes, which generates a low to high pulse which is given to IR1 input of 8259A programmable interrupt controller (PIC). A distance of 100 meters is covered on 50 rotations of the bicycle wheel. Mask all interrupts other than IR1. Assume IR0 interrupt type is mapped to 90H.

a) Write ICW register values required in this interrupt based application.
b) Write OCW values required in this application.
c) Write ALP to initialize the interrupt controller using ICW and OCW..
d) Write ISR for the given interrupt type to count number of pulses and increment the distance Value by 100 meters. The distance in Meter is stored in word size memory location named MTR and distance in kilometer is stored in word sized memory location Named KM. ISR must exit with non specific end of interrupt (EOI) command.
e) Draw the schematic diagram of 8259A interfacing with 8086. It should clearly show address decoding and other required connections between processor and PIC. Assume 8259A base address as 740H.

Note: No need to interface Display device and its code snippet . **[20M]**



Q2. An 8086 microprocessor based system requires 32KB of ROM and 64 KB of RAM memory . The Size of Available ROM and RAM chips are 16 KB and 32KB respectively. Assume both types of memories are byte addressable. You are supposed to select the starting address of ROM suitably and the RAM address must start at 00000H. Assume contiguous memory locations for both ROM/RAM.
a). Write the entire range (memory map) of RAM/ROM addresses using the format given in the table.
b). Draw the complete **labelled** memory interfacing diagram using absolute addressing. use only NAND gates if required (you can choose variable multiple input NAND gates). Show all control signals. **[20M]**

Memory Chip	(Start Address in **Hexadecimal**)	(End Address in **Hexadecimal**)

Q3. You are supposed to design a 8086 based Smart Car parking lot system. The working of the system is as follows. There is a **display board** outside the parking lot which displays the number of vacant slots in the car park. At the entry/exit of the car park there are individual **sensors** which detects the entry/exit of a car . Also there are individual UID scanners at the entry and exit. As soon as a car enters, the **UID** (unique ID) of the car is scanned by a device and its entry time (T_IN) is noted. Once the car enters the parking, the available slot count on the display board is reduced by 1. Similarly, when a car exits, the UID and exit time(T_OUT) is noted and the the vacant slot count is incremented by one. The system maintains a real time clock in seconds which is used for recording the entry/ exit time. Assume that the 8-bit UID value of an incoming car is given by a parallel input device.
Based on the duration of the stay, the money to be charged is evaluated as C_sec * (T_OUT – T_IN). The C_sec is the cost of stay per second. The money to be charged from individual users are stored in the system. No need to display the charge.
Following components are given:
8086 – 1 No., 8255- 1 No., seven segment displays – 2 Nos., 7447- 1 No., logic gates (as per need).
a). Design the smart parking lot system using the given components. Show the interfacing diagram with 8086. For each of the chips (e.g. 8255) clearly highlight the mode, control words, etc. The base address of 8255 is 87H. If you require to use INTR interrupt use interrupt type 64H. However, there is no need to implement the circuitry to send this interrupt type to 8086.
b). Write the ALP for each of the required functionalities described in the question.
Constraints:
When a car comes and the sensor senses it, then the 8086 is intimated (note: 8086 should not be constantly scanning/ checking for the entry/ exit of a car as it might be engaged in some other tasks).
Assumptions:
Current time of the system (in seconds) is present in a word-sized memory location called **TIME**. For solving the problem, you can assume that the number of slots is in range 0 < no. of slots < 99. **[30M]**

Q4. Genetic algorithm for optimization uses various operations on binary coded values to optimize certain functions. One of these operations is **"Mutation"**. The mutation operation is implemented in such a way that it complements bits at two locations of a 16-bit binary number. The 16 bit numbers are stored at a location **"POPULATION"**. The locations, at which complementing has to be performed is stored at bytes sized location **"LOC"**. The upper and lower nibble for each data at **LOC** represents the two locations (from right) at which complementing has to occur (0 corresponds to LSB and F corresponds to MSB, see example). Write a well commented, 8086 based ALP for performing *MUTATION* operation and store the mutated population at Memory location **"MUTATED"**. The size of populations is 30. **[10M]**
Example:

POPULATION	LOC	MUTATED
2156H (0010 0001 0101 0110)	19H	**2354H** (0010 0011 0101 0100)

Voltage Amplifier
Current Amplifier
Transconductance Amplifier
Transresistance Amplifier

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
II SEMESTER 2018-2019
CS F241/EEE F 241/INSTR F 241 MICROPROCESSOR PROGRAMMING AND INTERFACING
COMPREHENSIVE EXAMINATION
PART I (CLOSED BOOK)

PART A Answer Sheet

NAME: **ID.NO:**

Q. No.	Answer	Q. No.	Answer
1	D	21	A
2	D	22	B
3	C	23	A
4	A	24	D
5	B	25	B
6	C	26	C
7	D	27	B
8	A	28	A
9	D	29	A
10	C	30	D
11	B	31	A
12	B	32	B
13	D	33	D
14	D	34	A
15	D	35	D
16	A	36	D
17	D	37	A
18	D	38	D
19	A	39	B
20	D	40	C

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
II SEMESTER 2018-2019
CS F241/EEE F 241/INSTR F 241 MICROPROCESSOR PROGRAMMING AND INTERFACING
COMPREHENSIVE EXAMINATION
PART I (CLOSED BOOK)

PART A Answer Sheet

NAME: **ID.NO:**

Q. No.	Answer	Q. No.	Answer
1	C	21	C
2	D	22	B
3	A	23	A
4	D	24	A
5	C	25	D
6	D	26	A
7	D	27	B
8	C	28	A
9	A	29	D
10	B	30	B
11	A	31	D
12	D	32	A
13	D	33	D
14	A	34	B
15	D	35	C
16	B	36	A
17	B	37	B
18	D	38	D
19	D	39	A
20	D	40	D

Design a bicycle odometer using 8086 microprocessor. Odometer keeps a record of total distance travelled by the bicycle (in Km). There is a magnet mounted on the wheel spokes as shown in the illustration, each time magnet passes through the rim as shown, a magnetic reed switch closes, which generates a low to high pulse to the IR1 input of 8259A programmable interrupt controller (PIC). IR0 interrupt type is 90H. A distance of 100 meters is covered on 50 rotations of the bicycle wheel. Mask all interrupts other than IR1.

- Write ICW register values required in this interrupt based application. [3]
- Write OCW values required in this application. [3]
- Write ALP to initialize the interrupt controller using ICW and OCW. [4]
- Write ISR for the given interrupt type to count number of pulses and increment the distance in Km by 100 meters after each set of 50 rotations. Assume kilometer is a word size memory location in RAM which stores the distance value. No need to store decimal point in kilometer but resolution must be of 100 meters. ISR must exit with non specific end of interrupt (EOI) command.[6]
- Draw the schematic diagram of 8259A interfacing with 8086. It should clearly show address decoding and other required connections between processor and PIC. Assume 8259A base address as 740H. A0 of 8259A is connected to A1 of 8086.[4]

.

- Write ICW register values required in this interrupt based application. **[1 mark each]**
  ICW1 - 13H
  ICW2 - 90H
  ICW3 - NOT REQUIRED
  ICW4 - 01H

- Write OCW values required in this application. **[1.5 mark each]**
  OCW1 - FDH
  OCW2 - 20H
  OCW3 - NOT REQUIRED

- Write ALP to initialize the interrupt controller using ICW and OCW. **[4 Mark]**

  1 Mark - Order of initialization
  2 Mark - Correct Address
  1 Mark - Correct use of OUT instruction

```
MOV AL, 13H    ;setting ICW1
MOV DX, 0740H
OUT DX, AL
MOV AL, 90H    ;ICW2
MOV DX, 0742H
OUT DX, AL
MOV AL, 01H   ; ICW4
OUT DX, AL
MOV AL, 0FDH ; OCW1
OUT DX, AL
```

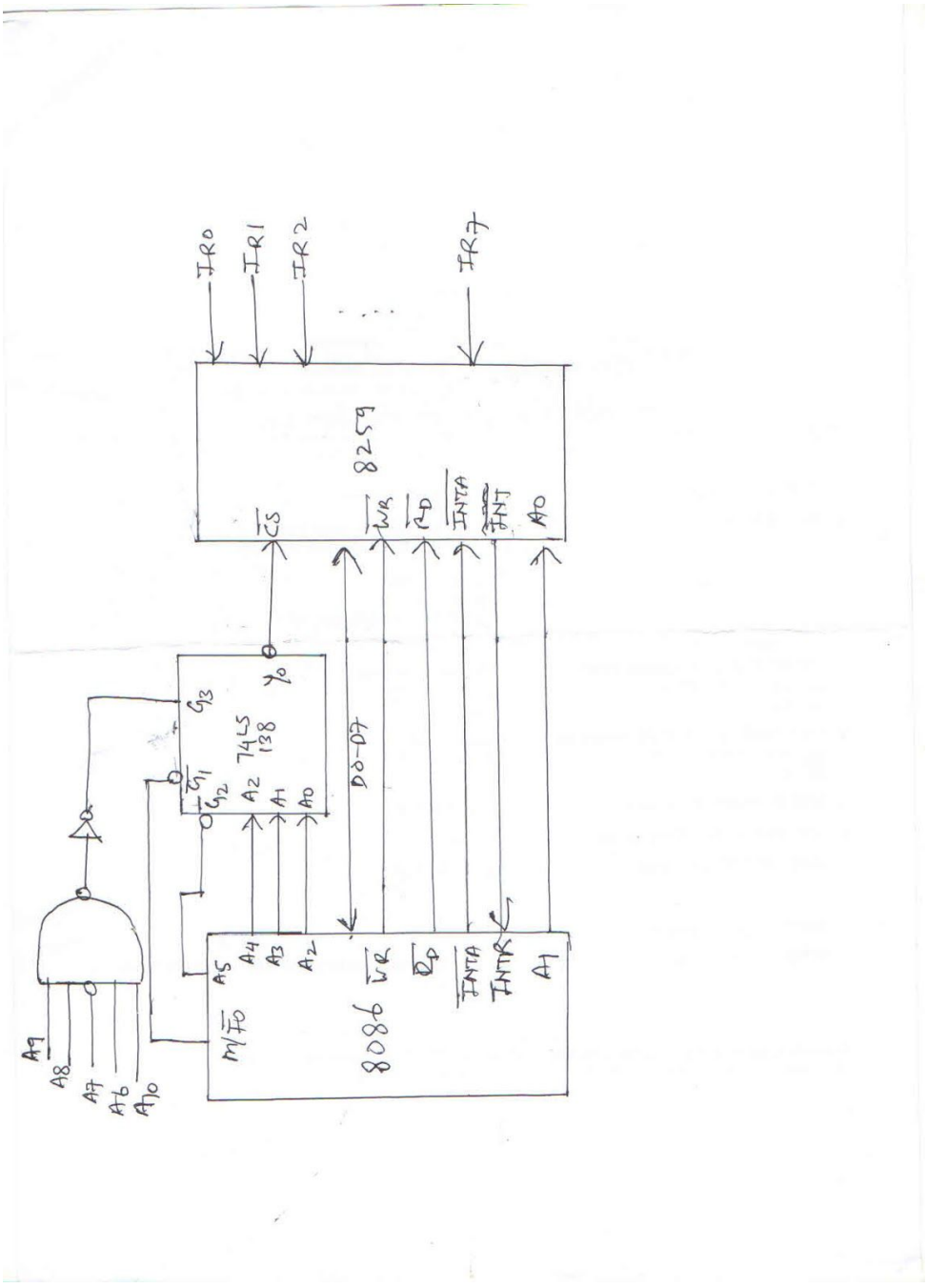
- Write ISR for the given interrupt type to count number of pulses and increment the distance by 100 meters after each set of 50 rotations. Assume distance is stored in a word size variable named 'mtr' in memory. **[6 Mark]**

  2 Mark for Correct EOI insertion
  4 Marks for overall logic,
  - 2 for incorrect usage of instructions
  - 4 for more than 2 instructions of different type are incorrect in the program

```
MTR DW ?
KM DW ?
TICK DB ?
ODOMETER PROC FAR
                LEA BX, TICK
                LEA CX, MTR
                LEA AX, KM
                INC [BX]  // Increment tick
                CMP [BX], 32H  // when tick reaches 50
                JAE DIST
                JMP IEXT

DIST:           MOV [BX], 0  // Making tick 0
                MOV DL, 64H
                ADD [CX], DL    / Increment MTR by 100
                CMP [CX], 3E8H // check if MTR is equal to 1000
                JE MTRRST
                JMP IEXT


MTRRST:         MOV [CX], 0 // Making MTR 0
                INC [AX]
                JMP IEXT

IEXT:
        MOV AL, 20H   // OCW2 for non specific EOI
        MOV DX, 0740H // Address of OCW2
```

```
        OUT DX, AL
        IRET

ODOMETER ENDP
```

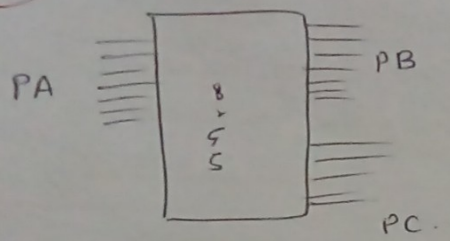
- Draw the schematic diagram of 8259A interfacing with 8086. It should clearly show address decoding and other required connections between processor and PIC. Assume 8259A base address as 740H. **[4 Marks]**
  2 Marks for correct address decoding (M/IO' to be used for decoding or IOR'/IOW' to be used. A16-A19 should not be used for decoding)
  2 Marks for all other connections are shown.
  - 2 if even one connection is missing (SP/EN also need to be connected to Vcc)



Connections above or any equivalent connections are fine.

8255 Addresses

P-1



Port A — 87h.
Port B — 8Fh
Port C — 97h
~~Porc~~Creg - 9Fh

4.

	A_4 A_3		
P-A	1000	0111	87h
P-B	1000	1111	8Fh
P-C	1001	0111	97h
Creg	1001	1111	9Fh.

Carentry → Port A (reads UID)

P.A mod CW's → 1 0 0 1 0 0 1 0 → 92H

```
porta   equ  87h
portb   equ  8Fh
portc   equ  97h
creg    equ  9Fh
mov     al, 92h
out     creg, al
```

2.

Programing/Initializing 8255.

Or

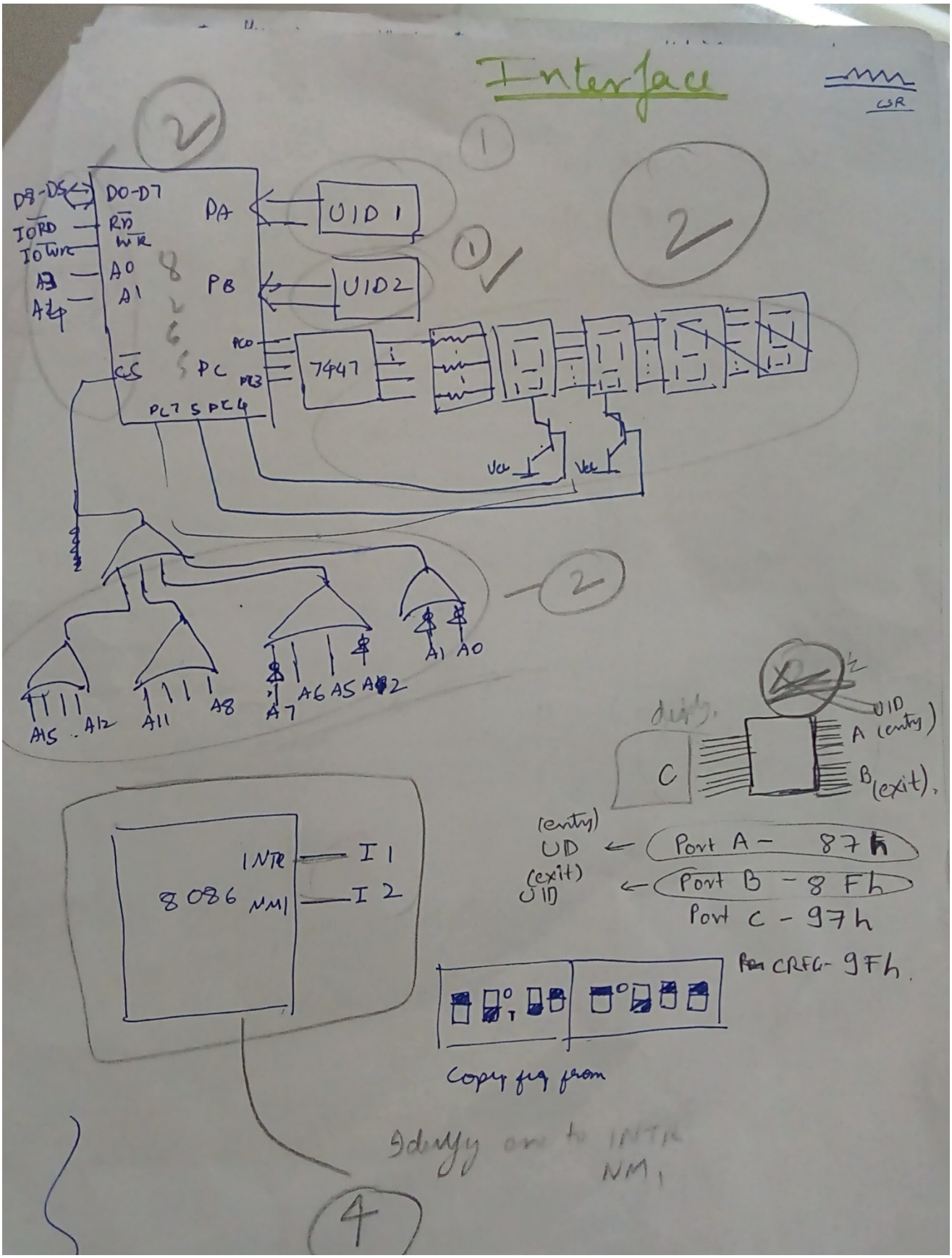
		$a_3a_2a_1a_0$
87h	1000	0111
89h	1000	1001
8Bh	1000	1011
8Dh	1000	1101

Note:
Marks will be given only if Interface drawn accordingly

a_3	a_2	$\overline{a_1}$
0	0	0
0	1	1
1	0	0
1	1	1

a_3	a_2	$\overline{a_0}$
0	0	0
0	1	1
1	0	0
1	1	1

Interface

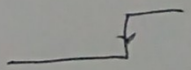

CSR
D0-D7
DO-D7
IORD
IOWR
A0
A1
PA
PB
PC
PC0
PC3
PC7
PC4
UID 1
UID2
7447
Vcc
Vss
A15
A12
A11
A8
A7
A6
A5
A4
A2
A1
A0
C
A (entry)
B (exit).
UID
(entry)
UID
(exit)
UID
Port A - 87h
Port B - 8Fh
Port C - 97h
for CREG - 9Fh.
INTR
NMI
8086
I1
I2
Copy fig from
4

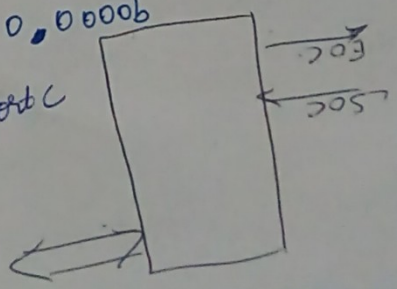
Code

(2)

```
.model tiny
.data
IN_ID   db   00H, 01H, ---- , 00H
TIME    dw   0000H
T_IN    dw   0000H, 0000H ---- , 0000H ; no. of elements equal to ID
T_OUT   dw   0000H, ----          0000H
OUTID_
FREE    db   00H, 00H ----        00H ; no. of elements equal to ID
DISPLAY dw   00H, 00H
COST    dw   0000H, 0000H ----    0000H ; equal to ID
C_SEC   db   03H
.code
.startup
CALL DISP7
.exit
DISP7 proc NEAR
PUSHA
XN2: MOV AX, DISPLAY
     OR AL, 1110,0000b
     OUT 97H, AL ; port C
     MOV CX, 1000H
XN:  NOP
     LOOP XN ; Software delay
     MOV AL, A1H
     OR AL, 1101 0000b
```

(3)






```
MOV CX, 1000H
XN1: NOP
     LOOP XN1
     JMP XN2
.exit
```

Code

```
10_ISR Proc far

Pusha
in Al, porta

MOV BX, OFFSET ID
ADD BX, FREE_index SI
MOV [BX], AL

MOV AX, TIME

MOV BX, OFFSET T_IN
ADD BX, FREE SI
MOV [BX], AX
POPA
IRET

10_ISR ENDP
```

```
MOV AX, DISPLAY
DEC AX
ADD AL, 00H
CMP AL, 0FFH
JNZ X1
SUB AL, 06H
X1: MOV DISPLAY, AX
CALL DISP
ADD DI, SI
INC BYTEPTR [DI]
; make 0FH in free array
```

```
MOV SI, OFFSET FREE
REPNE MOV AL, 00H
REPNE CMPSB
MOV DI, OFFSET FREE
SUB SI, DI
```

```
I1_ISR proc FAR
PUSHA
IN AL, port B
MOV SI, OFFSET ID
MOV DL, AL
REPNE CMPSB

MOV DI, SI
LEA BX, ID
SUB SI, BX
LEA BX, T_IN
ADD BX, SI
MOV AL, [BX]
MOV AH, TIME
```

```
MOV AL, AH
MOV DH, C_OUTSEC
MOV AH, 00H
MUL DH
LEA BX, COST
ADD BX, SI
MOV [BX], AX
CALL DISP

MOV AX, DISPLAY
INC AX
CMP AL, 0AH
JNZ X2
INC AH
MOV AL, 00H
MOV DISPLAY, AX
CALL DISP
POPA
```

Q.4

=> [DATA DECLARATION]

```
.code
.startup
        LEA DX, LOC
        LEA DI, Mutated
        LEA SI, Population
        MOV CH, 30
X1:     MOV CL, Byte ptr [DX]
        AND CL, OF H
        INC CL
        MOV AX, 00
        STC
        RCL AX, CL
        MOV BX, Word ptr [SI]
        XOR BX, AX
        MOV CL, Byte ptr [DX]
        AND CL, 0F0H
        SHR CL, 04 H
        MOV AX, 00
        INC CL
        STC
        RCL AX, CL
        XOR BX, AX
        MOV word ptr [DI], BX
        INC DI
        INC DI
        INC SI
        INC SI
        INC DX
        DEC CH
        JNZ X1
.exit
        END
```

NOTE:- If 'A' is not correct, (B) & (C) is not awarded marks, same goes for part 'B' & 'C'.

Marking scheme:-

A. (2m) → separating out nibbles correctly

B. (5m) → complenting bits correctly.

C. (3m) → storing result & looping correctly.